

Apunte de la unidad 2: Testing y Errores

Programación con Objetos 1



Unidad 2: Testing y errores

2.1 Pruebas (Testing)

2.1.1 Definición de un test en WolloK

2.1.2 Ejecución de los tests

2.1.3 Testeando acciones

2.1.4 Interfaz del objeto assert

2.1.5 El método initialize

2.1.6 Algunas características de los tests Automáticos

Aislado

Automático

Repetible

2.1.7 Tipos de test automáticos

2.1.8 Desarrollo guiado por pruebas (TDD)

2.1.9 Diseño de casos de prueba

2.1.10 Conclusión

2.2 Errores (Excepciones)

2.2.1 Conceptos básicos de errores

2.2.2 Errores de programa y de usuario

2.2.3 Mejorando la calidad de las validaciones.

2.2.4 Estado interno

2.2.5 Atrapar errores

2.2.6 Testear errores

2.2.7 Rollback

2.2.8 Malas prácticas

2.2.8.1: Mala práctica: Ignorar el problema.

2.2.8.2: Mala práctica: Programación defensiva.

2.2.8.3: Mala práctica: Chequeo por valor de retorno

Unidad 2: Testing y errores

2.1 Pruebas (*Testing*)

Al comenzar el estudio de la disciplina de la programación, los primeros programas son probados manualmente. Es decir, para determinar si un programa es correcto o no, una persona usuaria lo ejecuta y evalúa que la respuesta generada por el sistema sea la que está esperando.

Esta estrategia, si bien es útil en un principio, rápidamente encuentra un techo que la hace inviable, al menos como única estrategia: en un programa que solo es probado manualmente, es razonable no querer realizar modificaciones ni agregar funcionalidad por temor a introducir errores (también llamados *bugs*). Por otro lado, el tiempo que consume realizar una prueba manual de todo un sistema es un factor importante.

Por otro lado, la industria requiere que el ciclo de vida de los programas sea largo: se inicia con una versión inicial, y se sigue agregando y modificando funcionalidades en versiones nuevas que van reemplazando la anterior. El cambio es intrínseco al desarrollo tecnológico y no es deseable un programa poco mantenible.

En este sentido, las herramientas de versionado de código, tales como git, son una ayuda importante al momento de revertir cambios que introducen errores en el programa. Sin embargo, sigue siendo complicado determinar si el programa funciona según lo esperado y, en caso de que tenga errores, identificar dónde se debe corregir. Las pruebas automáticas, tema central de este documento, son la herramienta que soluciona el problema: permiten rápidamente, y en cualquier momento, determinar si el programa está funcionando como se espera.

La idea es la siguiente: el diseño de cada programa es acompañado por otro programa de prueba o test. El programa de test (que puede ser en realidad un conjunto de programas) tiene la capacidad de ejecutar el programa objetivo (aquel que se está evaluando) y luego determinar si el resultado de la ejecución es lo que se esperaba. Si se concluye que (el programa objetivo) funciona correctamente, entonces es adecuado publicarlo en el repositorio. En caso contrario se debe trabajar sobre los errores encontrados y repetir la prueba, es decir: ejecutar nuevamente el programa de test.

Es importante ver que el programa de test evoluciona con el programa objetivo. Es decir que si se necesita agregar funcionalidad o modificar el comportamiento de este último (asumiendo que estaba probado y correcto), se deben acompañar con modificaciones en el programa de test, probablemente incorporando nuevos casos de prueba (o bien ajustando los casos que ya estaban definidos). De esta manera es posible detectar inmediatamente si la nueva funcionalidad o las modificaciones han invalidado algo que ya funcionaba. Si el programa de test determina que no hay errores, entonces se lo integra al repositorio. En caso contrario –como antes– se debe encontrar el origen del error, corregirlo y volver a ejecutar las pruebas.

2.1.1 Definición de un test en Wollok

Los programas de test se definen en el mismo proyecto en el que está el programa objetivo, pero en archivos independientes. Se debe crear un archivo con extensión `.wtest`, con un nombre

autoexplicativo diferente al de los otros archivos `.wlk` donde se encuentran las definiciones. Si se usó el comando `wollok init` para inicializar el proyecto, el mismo ya creó un archivo de tests llamado `testExample`.

A continuación, un ejemplo de un archivo de test, que incluye un único test.

```
import pepita.*
describe "Tests de Pepita" {
  test "energía inicial de pepita" {
    assert.equals(100, pepita.energia())
  }
}
```

La estructura de estos archivos es como sigue. Al inicio, se necesita hacer el `import` para incluir el código a testear, indicando el nombre del archivo donde se encuentran definidos los objetos. Se recomienda poner `nombreDeArchivo.*` para contemplar todas las entidades definidas en dicho archivo. Los tests suelen estar agrupados en conjuntos denominados *describe*. Si bien es opcional el *describe*, es una buena práctica con algunas ventajas que se explicarán más adelante. Se utiliza un nombre expresivo para identificarlo, que sigue las convenciones de los strings, por lo que se usan comillas dobles. Al igual que los objetos, se utilizan llaves `{ }` para delimitar el inicio y fin del conjunto de tests. Cada test comienza con la palabra reservada `test` seguida de una cadena de caracteres que explique lo que se está probando. Es importante utilizar una buena descripción porque esa misma leyenda es la que va a aparecer en el informe de errores, y cuando el test falle permitirá detectar más fácilmente cuál fue el problema: es bueno identificar el escenario de prueba y no entrar en detalles redundantes (por ejemplo, decir que la energía queda en 85 implica que si cambian las reglas de cálculo debemos cambiar en dos lugares).

Por último, el elemento clave es un objeto autodefinido de la biblioteca `lib` denominado `assert` que entiende los mensajes que permiten plantear las expectativas declarativamente, delegando internamente al framework de testing que provee Wollok toda la lógica de la ejecución y validación automatizada. Es decir que se le envía un mensaje a este objeto para realizar concretamente la comparación, contrastando el resultado de algún mensaje con el valor esperado.

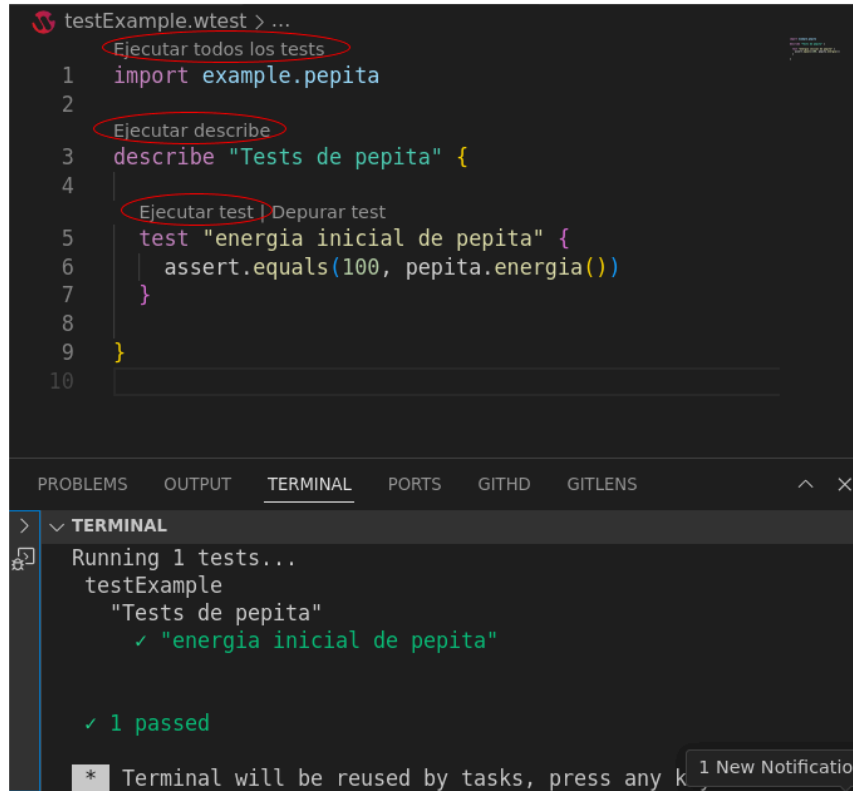
Retomando el ejemplo de pepita, el primer caso de prueba podría plantearse de esta manera, donde se utiliza el método `equals()` del objeto `assert` para evaluar por condición de igualdad la energía del objeto pepita con la constante 100, que es el valor esperado. Este test, entonces, está verificando que la energía inicial de pepita es 100.

2.1.2 Ejecución de los tests

Hay distintas maneras de ejecutar los tests, siendo la más sencilla utilizando el IDE. Se pueden ejecutar todos los tests del proyecto, todos los tests de un *describe* o un test individual. La manera es similar a la ejecución del REPL: un link por encima de las definiciones en los archivos. En la terminal se ve el resultado.

La ejecución de los tests tiene 3 posibles resultados: que el test (o conjunto de tests) sea exitoso, que haya fallas en las validaciones o que haya fallas en los objetos evaluados.

La siguiente imagen describe un caso exitoso



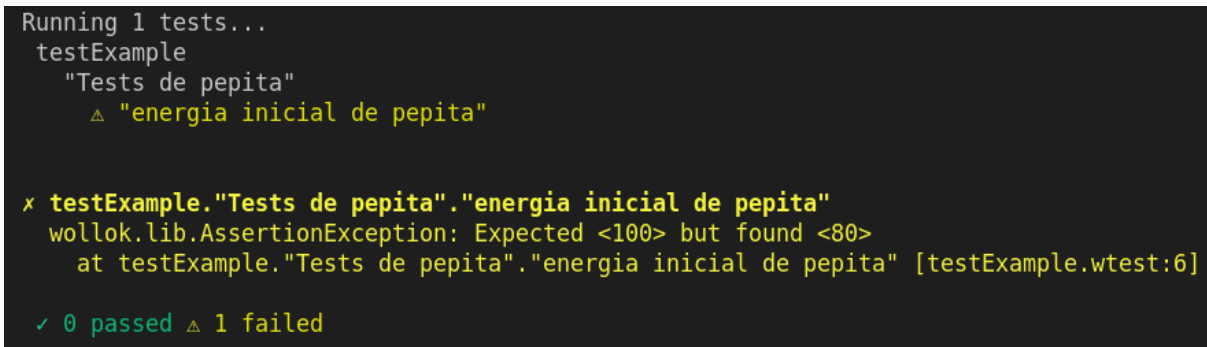
```

testExample.wtest > ...
1  import example.pepita
2
3  describe "Tests de pepita" {
4
5    test "energía inicial de pepita" {
6      assert.equals(100, pepita.energia())
7    }
8  }
9
10

PROBLEMS  OUTPUT  TERMINAL  PORTS  GITHD  GITLENS
>  TERMINAL
Running 1 tests...
testExample
  "Tests de pepita"
    ✓ "energía inicial de pepita"

✓ 1 passed
  
```

En la siguiente imagen se describe un fallo en la validación:



```

Running 1 tests...
testExample
  "Tests de pepita"
    △ "energía inicial de pepita"

x testExample."Tests de pepita"."energía inicial de pepita"
  wollock.lib.AssertionException: Expected <100> but found <80>
    at testExample."Tests de pepita"."energía inicial de pepita" [testExample.wtest:6]

✓ 0 passed △ 1 failed
  
```

El informe de ejecución de tests tiene toda la información necesaria para corregir errores. Esto es: qué test falló, la cantidad de fallas, por qué falló y en qué línea está ubicado el assert que falló. El ejemplo indica que se esperaba que la energía fuera 100, pero fue 80, y esto fue detectado por un mensaje ubicado en la línea 6 del archivo `testExample.wtest`. Por este mensaje y por la línea que valida el assert es posible suponer que el problema está en el valor que se utiliza al inicializar la energía de pepita. Probablemente el código dice: `var energia = 80` en lugar de 100.

Por último, si se produce un error durante la ejecución de los objetos probados, se aprecia un mensaje como el de la siguiente imagen:

```
Running 1 tests...
testExample
  "Tests de pepita"
    x "energía inicial de pepita"

x testExample."Tests de pepita"."energía inicial de pepita"
wolok.lang.EvaluationError: Error: Could not resolve reference to enregia
  at example.pepita.energia() [example.wlk:5]
  at testExample."Tests de pepita"."energía inicial de pepita" [testExample.wtest:6]

✓ 0 passed x 1 errored
```

En estos casos se dan detalles de la pila de ejecución (o *stacktrace* en inglés). La pila de ejecución describe la secuencia de invocaciones que se ejecutó hasta que ocurrió el error y es muy útil para poder repararlo. Si se quiere seguir el camino, se debe leer de abajo para arriba: en la línea 6 de `testExample.wtest` se llama a `pepita.energia()` que está en el archivo `example.wlk` en la línea 5. En ese lugar es donde ocurrió el problema: No pudo resolver la referencia “energía”. Al revisar la línea de código indicada, se puede detectar un error de tipeo: la variable está mal escrita, pues debería ser “*energia*”.

En este punto es importante destacar la diferencia entre **falla** y **error**. Una falla es el resultado de la evaluación de un mensaje del objeto `assert`: no encuentra el resultado esperado. Mientras que el error indica un error de ejecución que ocurriría más allá de los tests. Contrariamente a lo que culturalmente sugiere el uso de colores en la información por consola (color amarillo de las fallas y el rojo de los errores), suele ser mucho más complicado resolver una falla que un error. En el error se cuenta con el *stacktrace* para llegar a la línea donde ocurrió el problema. Mientras que en la falla se tiene que revisar todo el código para ver por qué el resultado no es el esperado.

2.1.3 Testeando acciones

El test que se desarrolla en el ejemplo anterior es extremadamente simple, pues solo evalúa que Pepita comience con 100 unidades de energía. Es decir que lo que se está evaluando es una consulta.

Cuando se considera la evaluación de órdenes o acciones, el test requiere más complejidad, pues primero se deben enviar los mensajes de acción (que generan efectos). Y luego se utilizan mensajes de consulta para confirmar que el efecto (de la acción) es el esperado.

```
import example.pepita
describe "Tests de Pepita" {
  test "energía inicial de pepita" {
    assert.equals(100, pepita.energia())
  }
  test "pepita vuela" {
    pepita.volar(5)
    assert.equals(85, pepita.energia())
  }
}
```

En la mayoría de los escenarios de prueba se requiere realizar un paso adicional de preparación del ambiente, que debe realizarse antes de la ejecución del o los tests. Algunas bibliografías separan fuertemente la creación/configuración de objetos de prueba y las acciones propiamente dichas, en lo que se denomina el patrón *Arrange-Act-Assert*, que describe una etapa de armado (*arrange*), una etapa de actuación (*act*) sobre los objetos y finalmente una etapa de verificación (*assert*) sobre el estado final de los objetos. Continuando el ejemplo, si se quisiera testear a partir de un cierto estado de pepita, se debe generar ese estado en la etapa de *arrange*.

```
test "pepita vuela" {  
  pepita.energía(50) // Arrange  
  pepita.volar(5)    // Act  
  assert.equals(35, pepita.energía()) // Assert  
}
```

En esa convención, es necesario mantener claramente separadas las acciones de las verificaciones (*assertions*). Pero la tecnología es suficientemente flexible para dejarlo a criterio de quien programa, ya que podría considerar tener varios ciclos *act/assert*:

```
test "pepita vuela" {  
  pepita.energía(50) // Arrange  
  
  pepita.volar(5)    // Act  
  assert.equals(35, pepita.energía()) // Asser  
  
  pepita.volar(10)   // Act  
  assert.equals(15, pepita.energía()) // Asser  
}
```

2.1.4 Interfaz del objeto assert

Hasta ahora hemos usado solo el método `equals()` del objeto `assert`, sin embargo se pueden distinguir, al menos tres situaciones: la comparación por igualdad, la validación de que ocurra algo y la validación de que no ocurra. En la primera se utiliza, como ya se vio en los ejemplos anteriores, el mensaje `equals(esperado, obtenido)`, con dos parámetros: el primero es el valor que se espera y el segundo es el resultado obtenido, el cual suele ser resuelto como el envío de un mensaje.

En la segunda situación se espera que la respuesta a un mensaje sea `true`, y por lo tanto se utiliza el mensaje `that()`, con un único parámetro que proviene del envío del mensaje cuya respuesta se espera que sea afirmativa. La tercera situación es la dual de la anterior, el mensaje `notThat()` que evalúa que la respuesta de un mensaje sea `false`.

Si bien los mensajes booleanos podrían ser resueltos con el `equals`, suele quedar mucho más clara la intención del mensaje a verificar si se utiliza `that` y `notThat`.

Este podría ser un *describe* que utiliza más ampliamente el objeto `assert`:

```
describe "Tests de pepita" {
  test "energía inicial de pepita" {
    assert.equals(100, pepita.energia())
  }
  test "pepita vuela" {
    pepita.energia(50)
    pepita.volar(5)
    assert.equals(35, pepita.energia())
  }
  test "pepita vuela muchos kilómetros, y ya no es fuerte" {
    pepita.volar(60)
    assert.notThat(pepita.esFuerte())
  }
  test "pepita vuela pocos kilómetros y se mantiene fuerte" {
    pepita.volar(10)
    assert.that(pepita.esFuerte())
  }
}
```

2.1.5 El método `initialize`

En algunas ocasiones un conjunto de tests necesita el mismo escenario inicial. Para evitar duplicar código, se puede escribir un método `initialize()` —no debe llevar parámetros y debe llamarse exactamente así— dentro del `describe`. En el siguiente ejemplo, se utiliza este método para que `pepita` inicie todos los tests con 50 unidades de energía. Además, se pueden definir constantes y variables. En este caso, se utiliza una constante `alimento`.

```
describe "Tests de pepita" {
  const alimento = alpiste
  method initialize() {
    pepita.energia(50)
  }
  test "pepita come" {
    pepita.comer(alimento)
    assert.equals(70, pepita.energia())
  }
  test "pepita come mucho" {
    pepita.comer(alimento)
    pepita.comer(alimento)
    assert.equals(90, pepita.energia())
  }
}
```


2.1.6 Algunas características de los tests Automáticos

La inclusión de los tests en el desarrollo de software se basa en que estos respeten 3 propiedades importantes. Deben ser **aislados**, **repetibles** y (valga la redundancia) **automáticos**. A continuación se detalla cada una de estas propiedades.

Aislado

Un test debe dar siempre el mismo resultado, ya sea que se ejecuta individualmente como si se ejecutara con el resto de los tests del *describe*, o con todos los tests del sistema.

Podría sospecharse que esto no puede asegurarse en un ambiente de objetos, ya que los objetos tienen estado interno. Por ejemplo, el test que compara la energía inicial de pepita con 100 podría fallar si previamente otro test la hizo volar. Sin embargo, la herramienta de testing de Wollok se encarga de regenerar/reiniciar todo el ambiente nuevamente antes de cada test. Es decir, el objeto pepita que ejecuta el test de energía inicial no es el mismo objeto pepita que ejecuta el test de volar. Es similar a ejecutar el REPL, usar a pepita, frenar la ejecución y volver a ejecutar el REPL.

Automático

Que un test sea automático implica que se ejecuta sin la observación de una persona que determine si pasaron o no. Esto permite ser integrado con otras herramientas automáticas: por ejemplo se puede configurar que ante un push en el repositorio git se ejecuten, y notifique si algo falló.

Por otro lado, con el objetivo de mantener la automaticidad saludable, es recomendable no realizar tests automáticos que introduzcan demoras en la ejecución. Por ejemplo, el siguiente código no es testeable por completo, pues la invocación del método **accion()** en el test hará que tarde mucho: el método **esperarMucho()** tiene un costo temporal alto.

```
method accion() {  
  obj1.preAccion()  
  obj2.preAccion()  
  self.esperarMucho()  
  obj1.postAccion()  
  obj2.postAccion()  
}
```

Sin embargo, puede analizarse como evaluar de otra manera (con otras aserciones) todo lo demás. Entonces lo que corresponde es reordenar el código para que **accion()** se divida en dos métodos que sí sean testeables, y por ende, puedan ser invocados en un test:

```
method accion() {  
  self.preAccion()  
  self.esperarMucho()  
  self.postAccion()  
}
```

```
}  
  
method preAccion() {  
    obj1.preAccion()  
    obj2.preAccion()  
}  
  
method postAccion() {  
    obj1.postAccion()  
    obj2.postAccion()  
}
```

De esta manera el test podría descomponerse en otros dos, evaluando una funcionalidad que si bien no es equivalente a la original, es una parte importante y permite llegar a conclusiones sobre código más atómico, evitando la espera.

```
test "test accion antes" {  
    pepita.preAccion()  
    assert.equals( "<lo que espera>", pepita.consultaSobreElEfecto())  
}  
test "test accion posterior" {  
    pepita.postAccion()  
    assert.equals( "<lo que espera>", pepita.consultaSobreElEfecto())  
}
```

Repetible

Que una prueba sea repetible implica que cada vez que se ejecute se obtengan los mismos resultados. Para asegurar esto se deben evitar situaciones donde:

- se utilizan números u otros objetos aleatorios.
- el código depende de la fecha actual
- se accede a otros sistemas

Cada una de estas situaciones cuenta con una técnica que permite reemplazar el código no repetible por código predecible en la ejecución del test. Por ejemplo, el siguiente código no es predecible, por lo tanto no es testeable:

```
object pepita {  
    var property energia = 100  
    method energia() {  
        return energia  
    }  
    method volar(distancia) {  
        energia = energia - distancia - 10  
    }  
}
```

```
method volarLoQueSeTeAntoje() {  
  const cuanto = 1.randomUpTo(10) //un random de 1 a 10  
  self.volar(cuanto)  
}  
}
```

Pero es posible modificarlo mediante el uso del polimorfismo y creando un objeto maqueta (*stub* o *mock*) que permita testear el objeto. La diferencia entre *stub* y *mock* no es útil para este documento, pero lo importante es que es un objeto que reemplaza el usado normalmente con fines de testing. En el ejemplo, se encapsula la aleatoriedad del método **randomUpTo()** en un objeto **dadoEquilibrado** que puede ser reemplazado (para el escenario de prueba) por otro objeto **dadoCargado** que implementa el mismo método polimórfico **tirar()** y que es el que se usa en el test. El código del objeto a testear es entonces:

```
object pepita {  
  var property energia = 100  
  var property dado = dadoEquilibrado  
  method energia() {  
    return energia  
  }  
  method volar(distancia) {  
    energia = energia - distancia - 10  
  }  
  method volarLoQueSeTeAntoje() {  
    const cuanto = dado.tirar()  
    self.volar(cuanto)  
  }  
}  
object dadoEquilibrado {  
  method tirar() {  
    return 1.randomUpTo(10) //un random de 1 a 10  
  }  
}  
object dadoCargado {  
  method tirar() {  
    return 3 //siempre devuelve 3  
  }  
}
```

De esta manera el test se puede realizar así:

```
test "pepita vuela lo que se le antoja" {  
  pepita.dado(dadoCargado) // Le configuro el stub  
  pepita.volarLoQueSeTeAntoje() //Se que voló 3
```

```
assert.equals(87, pepita.energia()) //verifico el efecto sabiendo
                                   // que se le antojó 3
}
```

Por último, si el test modifica archivos o bases de datos, debe revertirlo al finalizar, para garantizar que si vuelve a correr está el ambiente tal como estaba originalmente.

2.1.7 Tipos de test automáticos

Dentro de un proyecto de software se pueden encontrar varios tipos de tests automáticos. En el contexto de esta materia, nos interesa entender los test unitarios y los test funcionales.

Test unitario: Verifica el comportamiento de una unidad mínima de código. Generalmente en objetos se asocia un archivo con tests automáticos dedicados a un único objeto (o tipo de objeto), y cada test se enfoca en una funcionalidad particular. Por ejemplo, en un único archivo se deberían definir todos los tests correspondientes al objeto pepita, en otro todos los tests de alpiste y en otro los de manzana. Se intenta aislar el objeto de sus dependencias, reemplazándolas por objetos stubs o mocks de ser necesario. La ventaja de tener test unitarios es que ante la falla de un test se acota la búsqueda del problema. Generalmente es muy recomendable tener este tipo de tests en un proyecto. En el contexto de esta materia no vamos a utilizarlos mucho debido a que el desarrollo de los mismos suele llevar demasiado tiempo.

Test funcional: valida que una funcionalidad del sistema cumpla con un requerimiento de negocio específico sin importar cómo esté implementada internamente. Los tests que más vemos en la materia están en este universo, ya que solemos escribir los mismos para ver cómo interactúan los objetos entre sí para resolver un requerimiento.

2.1.8 Desarrollo guiado por pruebas (TDD)

El desarrollo guiado por pruebas (*Test-driven development* - TDD) es una práctica de programación que requiere diseñar la solución a partir de pensar los casos de prueba y a partir de ello guiar el diseño y la programación (en este caso de los objetos). Generalmente estas pruebas pueden ser unitarias y/o funcionales.

En primer lugar, se escribe un caso de prueba y a continuación se escribe el código que hace que la prueba pase satisfactoriamente: definición de los objetos y métodos a demanda del caso de prueba. El propósito del *desarrollo guiado por pruebas* es lograr un código *limpio* que funcione, haciendo que los requisitos sean traducidos a pruebas, de este modo, cuando las pruebas pasen se garantizará que el software cumple con los requisitos que se han establecido.

Otra ventaja del TDD se da cuando uno no tiene muy claro cómo resolver un requerimiento. Entonces comenzar a escribir el test permite ir definiendo qué mensajes debe entender el objeto, guiando así la solución.

2.1.9 Diseño de casos de prueba

Independientemente del método elegido (pruebas manuales o automáticas), lo primero a considerar es qué casos se necesitan probar. En TDD, antes de tener el código escrito y conociendo los detalles

del problema a resolver, se puede identificar un abanico de situaciones posibles, que incluye desde los casos típicos y frecuentes hasta las situaciones límite o de escasa probabilidad, pero que de todas maneras también deben ser contempladas.

Es importante distinguir dos momentos: el diseño de las pruebas y la ejecución de las pruebas. Durante el diseño de pruebas se debe poder **anticipar**, para cada posible situación, cuál es el resultado correcto que se espera obtener. Mientras las pruebas se diseñan al comienzo (aunque pueden aumentarse y agregarse casos), se las ejecutan muchas veces, cada vez que se cambia algo en el código que se valida.

En particular, en el paradigma de la programación orientada a objetos, ¿Qué aspectos se quieren probar de un objeto? Principalmente, su comportamiento: queremos enviar mensajes al objeto y verificar que suceda lo esperado.

Considerar el ejemplo del ave Pepita, siendo el problema como sigue:

"Sabemos que Pepita tiene inicialmente 100 unidades de energía. Cuando come alpiste, su energía se incrementa en 20 unidades. Al volar, su energía disminuye en una unidad por cada kilómetro que vuela más 10 unidades fijas. Queremos comprobar si pepita es fuerte, lo cual sucede cuando su energía supera las 50 unidades, y también nos interesa saber en cualquier momento cuál es su energía actual".

En cuanto a las consultas que pueden hacerse al objeto pepita, se requiere que

- En la situación inicial de Pepita, es decir, sin que haya volado ni comido, la respuesta a la pregunta por su energía debe ser 100.
- En las mismas condiciones, si se le pregunta si es fuerte, la respuesta debería ser afirmativa. ($100 > 50$)

Esto describe al menos esos dos casos de prueba. Por otro lado, se deben considerar también los requerimientos que no son consultas sino que son acciones que deben provocar un cambio en el objeto. En este ejemplo se pueden destacar otros casos de este tipo:

- Al decirle a Pepita que vuele N km, asumiendo que estaba en su estado inicial, su energía debe ser $e' = 100 - (N + 10)$. Si $N=5$, entonces $e'=85$.
- Si Pepita, en su estado inicial, recibe la orden de comer alpiste, su energía debe ser $e' = 100 + 20$. entonces = 120.

Podríamos querer probar qué sucede con otros valores, por ejemplo, si Pepita volase otra cantidad de kilómetros (N) o comiese otro alimento. Plantear casos de prueba con todos los valores posibles es impracticable e irrelevante, sin embargo, es recomendable considerar ciertos valores que representen situaciones particulares, normalmente ubicadas en los límites de los datos involucrados (casos de borde). Por ejemplo, sería interesante pedirle a pepita que vuele 0 km (que podría significar que arranca a volar y se arrepiente instantáneamente) o que coma un alimento que aporta 0 gramos (que, aunque parezca trivial, también es una orden válida). Estos son casos especiales o de borde que ayudan a garantizar que el objeto funcione correctamente en todos los casos. Así se obtienen dos casos de prueba más:

- Al decirle a Pepita que vuele 0 kms, y asumiendo que estaba en su estado inicial, su energía debe ser 90. $(100 - (0 + 10) = 90)$
- Si Pepita, en su estado inicial, recibe la orden de comer un alimento que aporta 0 de energía, su energía sigue siendo la inicial de 100. $(100 + 0 = 100)$

Otro aspecto a probar es la condición de si Pepita es fuerte. Para probarlo no alcanza con la situación inicial porque se sabe que al comenzar es fuerte. Entonces lo que tenemos que hacer es preguntarle si es fuerte habiendo previamente volado una suficiente cantidad de kilómetros. Se tiene así 2 casos de prueba: cuando la energía queda por debajo del umbral y cuando es exactamente el valor del umbral (50).

- Partiendo de la situación inicial, si hacemos que Pepita vuele 60 kilómetros de manera que su energía llegue a 30 $(100 - (10 + 60))$, ante la pregunta si es fuerte, la respuesta debe ser negativa $(30 < 50)$.
- Partiendo de la situación inicial, si hacemos que Pepita vuele 40 kilómetros de manera que su energía llegue a 50 $(100 - (10 + 40))$, ante la pregunta si es fuerte, la respuesta debe ser nuevamente negativa (porque no se cumple $50 > 50$).

De manera resumida, los casos de prueba son:

Descripción	Acciones	Verificación
Energía inicial	ninguna	pepita queda con energía 100
Fortaleza inicial	ninguna	pepita es fuerte
Vuelo	que pepita vuele 5 kilómetros	pepita queda con energía 85
Alimentación	que pepita coma alpiste	pepita queda con energía 120
Alimentación (borde)	que pepita coma alimento nulo	pepita queda con energía 100
Vuelo (borde)	que pepita vuele 0 kilómetros	pepita queda con energía 90
Perder fuerza	que pepita vuele 60 kilómetros	pepita no es fuerte
Perder fuerza (borde)	que pepita vuele 40 kilómetros	pepita no es fuerte

2.1.10 Conclusión

Un test, esencialmente, es una porción de código Wollok en la que se describe una determinada situación, mediante el envío de mensajes a los objetos correspondientes, y se especifica cuál es el resultado esperado.

En las pruebas unitarias, la estrategia es identificar unidades significativas del código y probar casos puntuales donde estas intervengan, en lugar de hacer una gran y extensa prueba de un sistema

completo. Cada prueba se enfoca en un caso de prueba, explicitando mediante código la vinculación entre las situaciones planteadas y los resultados esperados.

Si mediante una misma prueba se evalúan muchas cosas a la vez, cuando esta falla es más difícil saber cuál es el motivo del fallo. En cambio, si en una prueba se considera un solo aspecto (mensaje, interacción) y esta prueba falla, se tiene una mayor certeza sobre cuál es el problema a corregir. Por este motivo, es útil el carácter unitario de las pruebas. Los mismos principios de modularización y delegación de responsabilidades que caracteriza al paradigma de objetos, ayudan a que sea relativamente sencillo identificar las unidades sobre las que se van a definir las pruebas.

Además, existen pruebas funcionales, que ayudan en el diseño de la solución al trabajar sobre casos de uso. Cada prueba considera la ejecución de un caso de uso, descubriendo la interacción entre los distintos objetos.

Ambos tipos de test (unitarios y funcionales) se utilizan complementariamente. En el código de cada prueba (o *test*) se describe la situación a probar y se especifica el resultado esperado, entonces la validación es realizada por el ambiente de Wollok sin mediar la interpretación de la persona, logrando mayor velocidad y confiabilidad. A su vez, tratándose de código escrito, los tests quedan guardados al igual que el código que estos evalúan y cuando se desea, se pueden ejecutar todos de una vez, obteniendo un informe del resultado de cada uno de ellos. De esta manera, cuando en el proceso de desarrollo, al correr los tests se detecta algún problema y se corrige la solución, es sencillo volver a ejecutar todo el conjunto de pruebas. Además, si una solución que ya se probó que funciona correctamente se quiere modificar o reescribir por algún motivo, basta con correr nuevamente todos los tests para garantizar que sigue funcionando adecuadamente. Ante el aumento en la cantidad y variedad de tests, estos se pueden agrupar y organizar de diferentes maneras, para permitir variantes en su ejecución.

Cada test se concibe en forma independiente de cualquier otro test. La lógica de ejecución de tests parte del supuesto de que cada uno se corre a partir de la situación inicial del sistema, es decir, que el ambiente se reinicia antes de cada test, garantizando su total independencia. Además, ante la detección de un problema por parte de algún test, los demás pueden seguir ejecutándose sin inconvenientes. El informe final detalla cuáles tests se ejecutaron sin inconvenientes y en cuáles se detectaron problemas, diferenciando fallas (no se obtuvo el resultado deseado) y errores (no se pudo terminar de ejecutar la prueba). Tanto en las fallas como en los errores aparece información importante para resolver el problema.

2.2 Errores (Excepciones)

2.2.1 Conceptos básicos de errores

Como personas que programan, nos hemos encontrado varias veces con errores cuando nos equivocamos en alguna línea de código. por ejemplo, en el siguiente programa:

```
object pepita {  
  var energia = 100  
  method comer(comida) {  
    energia = energia + comida.energiaQueAporta()  
  }  
}  
object manzana {  
  var madurez = 1  
  const base = 5  
  method energiaQueAporta() {  
    return base * madures  
  }  
}
```

Si pedimos que Pepita coma la manzana desde el REPL, deberíamos encontrarnos con un problema, ya que en el método `energiaQueAporta()` de manzana cometimos un error al escribir la variable `madurez` (se escribió con `s` en lugar de `z`) y eso implica que esté intentando usar una referencia que no existe. Efectivamente, al ejecutar se muestra el error en la consola:

```
wollok:example> pepita.comer(manzana)  
X Evaluation Error!  
wollok.lang.EvaluationError: Error: Could not resolve reference to madures  
  at example.manzana.energiaQueAporta() [example.wlk:15]  
  at example.pepita.comer(comida) [example.wlk:5]
```

El error suele tener un **tipo** identificado con un nombre, que nos da idea de qué tipo de problema es. En este caso es `wollok.lang.EvaluationError`: wollok no pudo evaluar la sentencia que quiso ejecutar. Luego, se asocia un **mensaje** que explica un poco más sobre el problema. En este caso es *Could not resolve reference to madures*. Lo que significa que intentó usar una referencia “*madurez*” que no existe.

Considerando complementariamente el tipo de error y el mensaje es posible identificar cuál es el problema. Pero otro detalle muy importante es que el error tiene también la pila de ejecución (*stacktrace* que se mencionó en la sección anterior) que describe la secuencia de invocaciones que había ocurrido al momento de ocurrir el problema. La primera línea de la pila es la más importante, porque es la que indica donde realmente ocurre el problema: línea 15 de `example.wlk`, en el método `energiaQueAporta()`. Esta información es la que realmente nos va a ayudar a resolver el problema.

También se ven otros métodos involucrados en la ejecución, en este caso el método `comer(comida)` de `pepita`. Este es el mensaje que se disparó desde la consola. En nuestro caso, el stack es relativamente acotado, pero podría ser una secuencia de métodos más larga si la cadena de

mensajes fuese así lo fuera. Una de las cuestiones interesantes del *stack* completo es que sabemos que esos métodos fueron invocados pero no terminaron de ejecutarse correctamente. **El flujo de ejecución fue interrumpido**. Podemos comprobar que la energía de pepita no fue modificada, porque la asignación a una nueva energía se iba a producir luego de la suma de la energía actual con lo que devuelve el método `energiaQueAporta()`. Cómo éste fue interrumpido y nunca llegó a devolver nada, ni la suma si la asignación ocurrieron:

Estas características básicas de los errores no son exclusivas de Wollok, sino que cualquier lenguaje orientado a objetos lo tiene, con sus matices propios de la tecnología.

2.2.2 Errores de programa y de usuario

No solo los *bugs* producen errores. Hay otros errores con orígenes muy variados, como problemas en el hardware, acceso a archivos, acceso a bases de datos, apis sin autorización o con malas direcciones, y muchas otras que están ajenas a las cuestiones del usuario: Si no hay base de datos, el usuario no lo puede resolver por su cuenta. Solo puede esperar a que algún técnico lo resuelva para seguir usando el programa. Todo ese gran conjunto de errores ajenos al usuario son los que denominamos **Errores de programa**

Luego, podemos darnos cuenta que éste es un mecanismo muy útil para trabajar con otro origen de problemas muy particular, los llamados **errores de dominio o de usuario**. Estos errores son **lanzados** por la persona que programa intencionalmente cuando se da cuenta que no puede resolver el problema que se le está requiriendo. En el paradigma de objetos podemos afirmar que: **Si un objeto no puede cumplir con la responsabilidad asociada al método que está ejecutando, debe lanzar un error**. Se le denomina errores de usuario porque son problemas que surgen porque la persona usuaria del programa está pidiendo cosas que no se pueden resolver. El programa anda correctamente y lo que no se cumple es alguna precondition.

Acompañemos tantas definiciones rimbombantes con un ejemplo clarificador. Pepita consume energía al volar, eso ya lo hemos resuelto. Pero ¿Qué sucede si le piden que vuele una distancia para la cuál no tiene energía?. En nuestro modelo actual la energía queda negativa, dejando una pepita bastante inconsistente. Este es un caso donde aplica la regla de que debemos lanzar un error: le estamos pidiendo a pepita que vuele una distancia pero pepita no puede hacerlo. Por lo tanto no cumple la responsabilidad y debería lanzar un error.

En Wollok, la manera más simple de lanzar un error de usuario es utilizando el mensaje `error(mensaje)` que entienden todos los objetos. Así quedaría una versión posible del `volar(distancia)` de pepita que **valida** que pueda volar.

```
method volar(distancia) {
  const energiaQueGasta = 10 + distancia
  if (energia < energiaQueGasta) {
    self.error("No se puede volar " + distancia + " kms" )
  }
  energia = energia - energiaQueGasta
}
```

Notar que no es necesario poner el *else* en el *if*, ya en caso de cumplirse la condición el lanzamiento del error se corta el flujo de ejecución, impidiendo que la energía se modifique.

Aquí vemos un ejemplo de la ejecución:

```
wollok:example> pepita.energia()  
✓ 100  
wollok:example> pepita.volar(9999)  
✗ Evaluation Error!  
wollok.lang.DomainException: No se puede volar 9999 kms  
  at If [example.wlk:7]  
  at example.pepita.volar(distancia) [example.wlk:6]  
wollok:example> pepita.energia()  
✓ 100
```

2.2.3 Mejorando la calidad de las validaciones.

El código previo puede ser criticado. La claridad del código ha empeorado. Hay demasiadas líneas para el caso “excepcional” (el del error) mientras que quedó sólo una línea para el flujo “normal” del dominio (la asignación). Una buena técnica para disminuir este problema es extraer las validaciones a un método separado. Aprovechando que el flujo se corta si hay un problema, la llamada a la validación actúa como una especie de filtro: si el código supera la validación entonces tenemos la seguridad de que el código de dominio se puede ejecutar sin problema.

Por último, vamos a extraer la precondition que debe cumplirse para poder ejecutar la lógica normal a un método *booleano*. Es importante que los nombres no den lugar a duda de si se trata de un método de validación (que es una orden que puede lanzar error o no devolver nada) de los métodos de consulta booleanos. Por eso reforzamos utilizando la palabra *validar* en el primer caso, y un nombre que denote una consulta para lo segundo:

```
object pepita {  
  var property energia = 100  
  method volar(distancia) { //orden principal  
    self.validarVolar(distancia) //solo una línea para invocar la validación  
    //A partir de ahora la lógica de dominio  
    energia = energia - self.energiaQueGasta(distancia)  
  }  
  method validarVolar(distancia) { //método para validar: es una orden  
    // la condición es por la negación de la precondition  
    if (not self.puedeVolar(distancia)) {  
      self.error("No se puede volar " + distancia ) //Acá se lanza el error  
    }  
  }  
  method puedeVolar(distancia) { //Método booleano con la precondition  
    return energia >= self.energiaQueGasta(distancia)  
  }  
  method energiaQueGasta(distancia) {
```

```
        return 10 + distancia
    }
    method energia() {
        return energia
    }
}
```

2.2.4 Estado interno

Si un error es lanzado en una consulta, es innecesario preocuparse por el estado interno, ya que las consultas no tienen efecto. En cambio, si es lanzado desde una orden, se debe asegurar que el estado del objeto no se modifique, pues la ocurrencia de un error implica que *“no se pudo cumplir con la responsabilidad”*, lo cual no necesariamente debe cancelar la ejecución del programa. En este sentido, dejar el objeto inalterado y listo para recibir nuevos mensajes es lo más seguro para nuestro programa.

Por otro lado, realizar las validaciones al inicio del método es una buena práctica y podría considerarse una manera natural de asegurar que no hay efecto. Sin embargo, los errores pueden ocurrir dentro de los mensajes llamados dentro de este método (y que quizás sean de otros objetos). En esos casos es necesario conocer de antemano si el envío de un mensaje a un objeto es susceptible de lanzar un error de dominio. Si solo una acción puede fallar, podemos ordenar el envío de este mensaje primero y realizar el resto de las acciones infalibles después.

En el siguiente ejemplo vamos a incorporar otro objeto más para que use **Pepita**. De esta manera vamos a tener una cadena más grande de llamadas que implica que la validación esté dentro de otro objeto separado del que se está analizando. Así es como introducimos al objeto **Milena**, que modela a la persona responsable de Pepita, quien ante el mensaje `pasear()` la hace volar y luego le da de comer. Además, utiliza un objeto **camino** que lleva registro de la cantidad de distancia que lleva recorriendo en el paseo. Es decir que cuando Milena recibe el mensaje **pasear** debe realizar 3 acciones.

El orden en que se producen las 3 acciones es una decisión importante. El método **comer** debe ejecutarse después de **volar** por una cuestión de definición de dominio: pepita comerá luego de volar, por lo que no puede usar la energía que gana en la comida para volar la distancia requerida.

Como **volar** es el mensaje que puede lanzar un error de dominio (mientras que **comer** no), se puede asegurar que ese orden no traerá problemas: si no logra volar y la ejecución se interrumpe, entonces no intentará comer. El objeto Pepita quedará con el mismo estado que tenía.

Con respecto al recorrido de la distancia, es una acción que es, en principio, independiente de Pepita. Según la lógica de dominio, es aceptable que esto ocurra antes o después de los mensajes hacia Pepita. No obstante, esa es una acción que no va a lanzar error, mientras que el mensaje **volar** de Pepita sí lo puede hacer. Por lo tanto, corresponde hacerlo posteriormente:

```
object caminoPorElParque {
    var recorrido = 0
```

```
method recorrer(distancia) {
    recorrido = recorrido + distancia
}
method recorrido() = recorrido
}
object milena {
    const alimento = alpiste
    const ave = pepita
    const distanciaARecorrer = 5
    const camino = caminoPorElParque

    method pasear() {
        ave.volar(distanciaARecorrer) //Es el único que puede romper
        ave.comer(alimento)
        camino.recorrer(distanciaARecorrer)
    }
}
```

Para muchos de los problemas alcanza con ordenar adecuadamente el envío de los mensajes. Sin embargo, si hay más de una acción que puede interrumpir la ejecución, corresponde realizar todas las validaciones antes.

Agreguemos ahora un límite que impone el camino, de tal manera que Milena no pueda recorrer una distancia más grande que la del propio camino. Esto da lugar a dos posibles situaciones de error: el mensaje **volar** de pepita y el mensaje **recorrer** del objeto **camino**. La manera más ordenada de resolver eso de manera de no alterar el estado es anticipar todas las validaciones, sin importar que cada validación se ejecute doble: primero en conjunto en el objeto milena, y luego al realizar propiamente la acción.

```
object caminoPorElParque {
    var recorrido = 0
    const longitud = 100

    method validarRecorrer(distancia) {
        if (recorrido + distancia > longitud) {
            self.error("No se puede recorrer " + distancia + " porque supera el
límite del camino")
        }
    }
    method recorrer(distancia) {
        self.validarRecorrer(distancia)
        recorrido = recorrido + distancia
    }
}
```

```
    method recorrido() = recorrido
}
object milena {
    const alimento = alpiste
    const ave = pepita
    var distanciaARecorrer = 5
    var camino = caminoPorElParque

    method validarPasear() {
        ave.validarVolar(distanciaARecorrer)
        camino.validarRecorrer(distanciaARecorrer)
    }
    method pasear() {
        self.validarPasear()
        ave.volar(distanciaARecorrer)
        ave.comer(alimento)
        camino.recorrer(distanciaARecorrer)
    }
}
```

Con esto cubrimos la mayoría de los casos (y la completitud de los casos que vamos a ver en esta materia). Pero si por algún motivo realizar la validación es muy costoso como para que se quiera evitar ejecutarla dos veces, o el efecto de ejecutar una acción pudiese invalidar el resultado de la validación ya ejecutada de otra acción, hay que realizar un manejo de errores más complejo atrapando la excepción, como se explica en la próxima sección.

2.2.5 Atrapar errores

Hasta aquí se discutió cómo el lanzamiento de un error interrumpe el flujo normal de ejecución. A pesar de que la intuición indica que el programa se cancela, es posible describir dos situaciones donde esto no es así. Por ejemplo, si al interactuar con los objetos mediante la consola de REPL se obtiene un error, este ambiente sigue activo. Por otro lado, si un test se encuentra con un problema, el entorno de ejecución de pruebas de Wollok registra el fallo y continúa ejecutando los tests siguientes. En ambas situaciones se sugiere que existe un mecanismo por el cual un error que es *lanzado* puede ser *atrapado*. Hay una construcción del lenguaje que permite detectar el error y ejecutar un código en consecuencia. Este código es el flujo “*excepcional*” que se contrapone al flujo “normal”. El flujo normal es la ausencia del error, mientras que el flujo excepcional es el conjunto de instrucciones que se ejecutan luego de detectar el error. Es por este motivo que en la jerga de programación los términos “Excepción” y “Error” son utilizados indistintamente. Si bien algunas tecnologías distinguen estos términos bajo definiciones arbitrarias, en este contexto son equivalentes.

Previamente a la tarea de definir cómo atrapar un error y desarrollar el flujo excepcional, es importante pensar dónde se puede poner este código. Si bien cualquier método que forma parte del *stacktrace* tiene capacidad de atrapar una excepción, los lugares donde corresponde hacerlo son definidos por la *arquitectura de software* de nuestro sistema. Una definición muy sintética de arquitectura de software¹ es la estructura que define cuáles son los componentes del sistema de software (como la interfaz de usuario, la lógica de negocio, la estrategia de persistencia, etc.), cuáles son sus responsabilidades y cómo se relacionan. En un ambiente de programación orientado a objetos también se debe definir (o identificar) una arquitectura que le da marco a la ejecución de los objetos. Si los objetos son ejecutados desde el REPL, éste tiene una arquitectura donde se encajan de tal forma que estén disponibles para ser invocados por la consola. Si son ejecutados desde los tests, es el mismo Wollok en su SDK que ofrece la arquitectura para ejecutarlos.

Esta idea de que la arquitectura (del REPL o de los tests) provee un contexto para contener aquellos objetos que se definen para resolver un problema sugiere que también la arquitectura define cómo definir el flujo excepcional. Es por esto que el código de flujo excepcional no está en los objetos de la lógica de negocio, pues allí solo se deben lanzar errores que describen cuando no es posible cumplir con la responsabilidad especificada como lógica de negocio. Otra parte de la arquitectura se encargará de atrapar los errores y ofrecer el flujo excepcional. Por ejemplo, cuando el REPL atrapa el error imprime el mensaje junto al *stacktrace* y se queda esperando por una nueva orden. Mientras que el framework de testing contabiliza el test como *error*, imprime la información y continúa con el próximo test.

Siendo conscientes de que es algo que no se suele hacer en el dominio (salvo muy contados casos). Podemos escribir un programa Wollok que utilice la construcción *try/catch* para entender cómo actúan estas otras partes del código.

Un programa wollok es un archivo *.wpgm*, con una estructura *program* que oficia de lo que en otras tecnologías se conoce como *main*. Cuando se ejecuta un *program* no estamos dentro de una arquitectura como en el REPL o en los tests. Por lo que si no hacemos nada en especial, un error efectivamente cancelará el programa.

```
program.wpgm
import example.*
program milenaProgram {
  pepita.energia(5)
  milena.pasear()
}
```

Al ejecutar el program se ve cómo se cancela:

 **Uh-oh... Unexpected Error!**
No se puede volar 100
at If [example.wlk:42]

1

https://es.wikipedia.org/wiki/Arquitectura_de_software#:~:text=La%20arquitectura%20de%20software%20define,computadora%5D%20tendr%C3%A1%20asignada%20cada%20tarea.

```
at example.pepita.validarVolar(distancia) [example.wlk:41]
at example.milena.validarPasear() [example.wlk:24]
at example.milena.pasear() [example.wlk:28]
at program.milenaProgram [program.wpgm:16]
```

* The terminal process `"/usr/bin/bash '-c', '/home/leo/Downloads/wollok' run 'program.milenaProgram' --skipValidations --port 4200 -p '/home/leo/workspaces/obj1unq/202502/pepita'"` terminated with exit code: 21.

El exit code: 21 denota que para el sistema operativo el programa terminó con un error.

Incorporemos entonces la estructura *try/catch* para poder atrapar la excepción y escribir el flujo excepcional.

```
program milenaProgram {
    try {
        pepita.energia(5)
        milena.pasear()
        console.println("Se paseó bien!")
    }
    catch e {
        console.println(e.message())
    }
    console.println("Se terminó")
}
```

Lo que está en el bloque `try{}` es lo que se considera flujo normal. Si en algún momento de ese flujo ocurre un error entonces se entra en el flujo excepcional que es lo que se escribe dentro del bloque `catch`.

```
No se puede volar 10
Se terminó
✓ Program program.milenaProgram finalized successfully
```

En nuestro caso cuando llama a `milena.pasear()`, éste llama a `pepita.volar(10)` que a su vez llama a la validación. En ese punto se corta el flujo normal, y la excepción comienza a *burbujear* por el *stacktrace* esperando llegar a un bloque `try` para saltar al `catch`: No encuentra `try` en el `pepita.validarVolar(distancia)`, no encuentra `catch` en el `pepita.volar(distancia)`, tampoco encuentra `catch` en el `milena.pasear()`, y al llegar al `program` sí encuentra un bloque `try`. Por lo que **evita** la línea que imprime “Se paseó bien” para ir a la línea que imprime el mensaje del error. Notar que `e` es una referencia a un objeto que modela el error ocurrido. Con ese objeto podemos obtener el mensaje o incluso el *stacktrace*. Al terminar el *try/catch* (ya sea porque ejecutó todo el flujo normal, o porque

pasó por el flujo excepcional, se continúa con la ejecución del programa: se imprime "Se terminó". Notar que aunque el programa lanzó internamente una excepción, al ser **manejada** por él mismo éste termina satisfactoriamente para el sistema operativo.

Si no hubiera bloque try/catch, el programa se cancelaría como ocurrió en la versión previa.

2.2.6 Testear errores

Por defecto, si en un test ocurre un error, éste se considera no satisfactorio. Pero como vimos durante este apunte, el hecho de que Pepita lance un error cuando le piden volar más distancia de la que puede es el comportamiento esperado. Entonces la pregunta es: ¿Cómo decirle a Wollok que quiero probar que si a Pepita le piden volar mucho debe lanzar un error? Para esto, el objeto **assert** tiene un método `throwsException(bloque)` que sirve para indicar que se espera que la ejecución de un mensaje dispare un error. Es decir que es correcto que ese error se dispare, e incorrecto en caso contrario.

Pero escribir lo siguiente tiene un problema:

```
assert.throwsException(milena.pasear())
```

El problema es que ese código asume que el parámetro del mensaje **throwsException()** es el resultado de `milena.pasear()`. Pero `milena.pasear()` no es un mensaje que dé un resultado y, más aún, disparará un error cortando el flujo de ejecución e imposibilitando la llamada al método `throwsException`.

Entonces para resolver el problema, se envuelve el código a ejecutar entre llaves:

```
assert.throwsException({milena.pasear()})
```

Al usar las llaves se está pasando por parámetro un objeto que representa el código que queremos ejecutar. Eso permite que pueda ser ejecutado posteriormente por el método `throwsException` en un *contexto controlado*. El **throwsException** tiene dentro un try/catch con una lógica algo contraintuitiva, pues si el flujo normal termina de ejecutarse con éxito es que falló, en cambio si tuvo que ejecutar el flujo excepcional es que se está comportando como se espera:

```
describe "Tests de milena" {  
  test "milena puede pasear normalmente" {  
    milena.pasear()  
    assert.equals(105, pepita.energia())  
    assert.equals(5, caminoPorElParque.recorrido())  
  }  
  test "milena no puede pasear si pepita tiene poquita energia" {  
    pepita.energia(5)  
    assert.throwsException({milena.pasear()})  
    assert.equals(5, pepita.energia() )  
    assert.equals(0, caminoPorElParque.recorrido())  
  }  
}
```



```
test "milena no puede pasear más distancia que el camino" {  
  pepita.energia(99999)  
  milena.distanciaARecorrer(105)  
  assert.throwsException({milena.pasear()})  
  assert.equals(99999, pepita.energia() )  
  assert.equals(0, caminoPorElParque.recorrido())  
}  
}
```

Running 3 tests...

testExample

"Tests de milena"

- ✓ "milena puede pasear normalmente"
- ✓ "milena no puede pasear si pepita tiene poquita energia"
- ✓ "milena no puede pasear más distancia que el camino"

✓ 3 passed

Pero si tuviéramos algún problema y no se lanza algún error esperado:

testExample

Running 3 tests...

testExample

"Tests de milena"

- ✓ "milena puede pasear normalmente"
- ⚠ "milena no puede pasear si pepita tiene poquita energia"
- ✓ "milena no puede pasear más distancia que el camino"

✗ testExample."Tests de milena"."milena no puede pasear si pepita tiene poquita energia"
wollok.lib.AssertionException: Block {milena.pasear()} should have failed
at testExample."Tests de milena"."milena no puede pasear si pepita tiene poquita energia"
[testExample.wtest:13]

✓ 2 passed ⚠ 1 failed

2.2.7 Rollback

Volviendo al tema que dejamos pendiente en 2.2.4: Suponemos que por algún motivo no se puede ejecutar todas las validaciones antes, ya sea porque es muy costoso validar doble, o porque una acción podría invalidar la validación previamente ejecutada de otra acción, entonces corresponde utilizar el try/catch para poder revertir el efecto de una acción ejecutada luego de que otra falle. A eso se lo conoce como *rollback*.

```
object caminoPorElParque {  
  ...  
  method rollbackRecorrido(distancia) {  
    recorrido = recorrido - distancia  
  }  
  ...  
}  
object milena {  
  ...  
  method pasear() {  
    camino.recorrer(distanciaAREcorrer)  
    try {  
      ave.volar(distanciaAREcorrer)  
      ave.comer(alimento)  
    }  
    catch e {  
      camino.rollbackRecorrido(distanciaAREcorrer)  
      throw e //relanzo  
    }  
  }  
  ...  
}
```

Hay que tener mucho cuidado de volver a lanzar la excepción luego de producida, ya que el try/catch que utilizamos tiene como objetivo revertir el efecto, pero no es capaz de dar una solución al problema: efectivamente el objeto milena no puede cumplir la responsabilidad de pasear y debe romper (lanzar el error). Esta estrategia debería ser la última elegida porque es bastante más compleja. Sólo usarla cuando lo visto anteriormente no puede ser usado.

2.2.8 Malas prácticas

2.2.8.1: Mala práctica: Ignorar el problema.

Es muy importante que si un objeto no puede cumplir la responsabilidad lance error. El siguiente código mantiene el estado consistente pero no lanza error. En la jerga se le dice “*comerse la excepción*”. El problema se da en que quien envía el mensaje continúa como si la acción hubiera sido realizada. Con esta implementación de pepita, el test de milena falla, ya que ella va aumentar la

distancia recorrida pero no se va a modificar la energía de pepita. Este tipo de bugs es muy difícil de rastrear para ser resuelto.

```
object pepita {  
  ...  
  method volar(distancia) {  
    //NO HACER ESTO! se come la excepción  
    if (self.puedeVolar(distancia)) {  
      energia = energia - self.energiaQueGasta(distancia)  
    }  
  }  
  ...  
}
```

2.2.8.2: Mala práctica: Programación defensiva.

El código debe mantenerse lo más simple y claro posible. Los objetos deben confiar en que sus colaboradores van a realizar sus propias validaciones, y que los argumentos que reciba van a ser razonables. En el dominio sólo hay que validar lo que tenga sentido para el dominio.

Algunos ejemplos de programación defensiva:

- validar que la distancia que vuela pepita sea positiva. Esto solo ensucia el código. Debe ser responsabilidad de la interfaz de usuario hacer una validación de este estilo.
- que milena chequee que pepita puede volar antes de enviar el mensaje volar (sin verdadera necesidad de hacerlo) Por ejemplo, en el caso que el camino no tenga validación, no existe motivo por el cual deba realizarse un chequeo antes del volar. Si había un problema el error se lanzará cuando corresponde
- que pepita chequee que el alimento que va a comer no sea nulo: Sólo ensucia el código, ya que si el alimento es nulo, va a romper solito el programa cuando le quiera enviar un mensaje

```
object milena {  
  method pasear() {  
    //Este if está mal! dejar que el ave.volar() rompa si no puede  
    if (not ave.puedeVolar(distanciaARecorrer)) {  
      self.error("No puedo pasear porque el ave no puede volar")  
    }  
    ave.volar(distanciaARecorrer)  
  }  
}
```

```
ave.comer(alimento)

}

}

object pepita {

  var property energia = 100

  method volar(distancia) {

    self.validarVolar(distancia)

    energia = energia - self.energiaQueGasta(distancia)

  }

  method validarVolar(distancia) {

    //Este if está mal! es responsabilidad de la UI darte valores
    válidos

    if (distancia < 0) {

      self.error("No se puede volar distancia negativa")

    }

    if (not self.puedeVolar(distancia)) {

      self.error("No se puede volar " + distancia )

    }

  }

  method comer(alimento) {

    //Este if está mal! por un lado es responsabilidad de la ui
    //Pero por otro lado, sin el if igual va a lanzar un error
    if (alimento != null) {

      self.error("No se puede comer un alimento nulo")

    }

    energia += alimento.energiaQueAporta()

  }

}
```

2.2.8.3: Mala práctica: Chequeo por valor de retorno

En tecnologías que no soportan lanzamiento de error que corte el flujo de ejecución, se suele usar el valor de retorno de los métodos para devolver si éste tuvo éxito o no. Puede ser un simple booleano o un código de error que luego hay que chequear. El problema de esto es que el código se vuelve más complejo. Todos los métodos deben estar preocupados por lo que ha devuelto las llamadas

internas para saber si puede seguir trabajando o no. Se mezcla lógica de manejo de error con lógica de dominio. Además sólo sirve para métodos de orden. Para métodos de consulta se va a volver muy complejo mezclar el valor de éxito/error con el valor de retorno de la consulta.

En su forma más sencilla: devolver un booleano en las acciones, se pierde el motivo del error, y el stacktrace.

```
program milenaProgram {
  if (milena.pasear())
    console.println("Se paseó bien!")
  else
    console.println("No se paseó bien pero no tengo idea de por qué")
}

object milena {
  ...

  method pasear() {
    return ave.volar(distanciaARecorrer) and
      ave.comer(alimento) and
      camino.recorrer(distanciaARecorrer)
  }
  ...
}

object pepita {
  ...

  method comer(alimento) {
    energia += alimento.energiaQueAporta()
    return true
  }

  method volar(distancia) {
    if (not self.puedeVolar(distancia)) {
      return false
    }

    energia -= self.energiaQueGasta(distancia)
  }
}
```

```
        return true
    }
}
```

Y si queremos saber cual es el problema, el código se vuelve notablemente más complicado:

```
program milenaProgram {
    const error = milena.pasear()
    if (error != null )
        console.println("Se paseó bien!")
    else
        console.println("Error: " + error)
}

object milena {
    ...

    method pasear() {
        var error = ave.volar(distanciaARecorrer)
        if (error != null) {
            return error
        }
        error = ave.comer(alimento)
        if (error != null) {
            return error
        }
        error = camino.recorrer(distanciaARecorrer)
        if (error != null) {
            return error
        }
        return null
    }
}

object pepita {
```

```
...  
  
method comer(alimento) {  
    energia += alimento.energiaQueAporta()  
    return null  
}  
  
method volar(distancia) {  
    if (not self.puedeVolar(distancia)) {  
        return "No puede volar"  
    }  
    energia -= self.energiaQueGasta(distancia)  
    return null  
}  
}
```

Se pueden encontrar más variantes sobre la misma idea. Esto solo puede tener sentido en una tecnología que no tenga capacidad de lanzar errores que corten el flujo.